



Identifying E-Commerce Fraud Through User Behavior Data: Observations and Insights

Ziyi Zhang^{1,2} · Hang Yin² · Susie Xi Rao³ · Xiao Yan⁴ · Zhurong Wang² · Weiming Liang² · Yang Zhao² · Yinan Shan² · Ruixuan Zhang¹ · Yuhao Lin¹ · Jiawei Jiang¹

Received: 31 July 2024 / Revised: 8 October 2024 / Accepted: 24 November 2024 / Published online: 15 January 2025
© The Author(s) 2025

Abstract

Traditional fraud detection approaches often use linking entities, such as device, email, and address, to identify fraudulent transactions and users. However, as fraud methods continue to evolve and escalate, the fraudsters can fabricate the involved entities and thus hide their real intent. To make fraud detection more robust, we incorporate user behaviors in the pipeline and consider biometric characteristics that are difficult to forge. In this work, we conduct a detailed study of how user behavior data can help identify and prevent fraudulent activity in e-commerce. We present Multi-Modal Behavioral Transformer (MMBT), where we combine both inner-page behavioral data, such as mouse trajectory, and inter-page behavioral data, such as page view sequences. We propose to construct mouse trajectory data as an image, treat each mouse position as a pixel in the image, convert the image into small patches, and hence transform the mouse trajectory into patch index sequences. Our experimental results on real-world data show that MMBT significantly outperforms baselines — the precision@recall = 0.1 increases by up to 7%. In addition, we have built an online pipeline to operationalize our model. In production, the 99th percentile latency is maintained below 500 milliseconds, allowing the platform to initiate rapid response measures and prevent potential losses.

Keywords Fraud detection · User behavior · Mouse trajectory

Ziyi Zhang and Hang Yin have contributed equally to this work.

✉ Jiawei Jiang
jiawei.jiang@whu.edu.cn

Ziyi Zhang
ziyi_zhang@whu.edu.cn

Hang Yin
hangyin@ebay.com

Susie Xi Rao
srao@ethz.ch

Xiao Yan
yanxiaosunny@gmail.com

Zhurong Wang
zhurowang@ebay.com

Weiming Liang
weimiliang@ebay.com

Yang Zhao
yzhao5@ebay.com

Yinan Shan
yshan@ebay.com

Ruixuan Zhang
zrx123@whu.edu.cn

Yuhao Lin
yuhaolin@whu.edu.cn

¹ School of Computer Science, Wuhan University, Wuhan, China

² eBay, Shanghai, China

³ Department of Computer Science, ETH Zurich, Zurich, Switzerland

⁴ Centre for Perceptual and Interactive Intelligence (CPII), Pak Shek Kok, Hong Kong, China

1 Introduction

In recent years, as the number of transactions using online e-commerce applications increases, financial fraud activities are growing at an astonishing rate, with worldwide losses caused by financial fraud up to billions of dollars each year [1]. Typical financial frauds in e-commerce applications include but are not limited to credit card fraud, insurance fraud, money laundering, healthcare fraud, and commodity fraud [2]. These fraudulent activities may disrupt economic stability, increase living costs, cause societal incidents, and reduce consumer trust in e-commerce platforms. For example, on e-commerce platforms, fraudulent accounts use stolen payment accounts to conduct bulk purchases, frequent orders, false refunds, and other improper behaviors, affecting the normal operations of the platform and causing victim merchants and consumers to suffer profit losses. Meanwhile, some fraudulent accounts are disguised by robots to steal users' private information, resulting in serious damage to the platform's reputation. Since the consequences of fraudulent activities in e-commerce applications are devastating, developing methods and technologies to detect and identify frauds is important and has attracted widespread attention.

Fraud detection has been studied in different scenarios, such as identifying malicious accounts [3–5]; anti-money laundering [6, 7]; spam reviews and news detection [8, 9]. Among these, we focus on transaction fraud detection [10–15], which is prominent in e-commerce platforms. If fraudulent transactions in practical applications can be identified, the e-commerce platforms can freeze malicious accounts and investigate legal responsibilities according to specific behaviors, thus effectively protecting

the interests of merchants and timely reducing economic losses.

To understand user behaviors, it is necessary to unfold the typical workflow of user actions on e-commerce platforms. Figure 1 illustrates a typical shopping procedure on an e-commerce platform. The user starts on the *main page*, entering keywords of the desired product in the *search bar*. Then, the user clicks the “Search” button, which leads he or she to the *search page*. On the *search page*, the user can see a panel or list displaying multiple products relatively (usually ranked w.r.t. relevance in descending order), along with their basic information such as name and price. Once the user selects and clicks a product, he or she will be directed to the *item page* where they can view more details about the product, including images, place of origin, and materials. Upon carefully reviewing the product information, the user then adds the product to the cart, proceeds to the *cart page* to enter delivery and payment details. After confirming all details are correct, the user can complete the purchase by clicking the “Confirm and pay” button. Note that, the user may jump between the pages, e.g., adding multiple items to the cart and then paying for them.

Although most users follow the aforementioned routine, their detailed actions regarding issues such as time consumption and page traversing trajectory can be significantly different. Figure 2 shows the action sequences of different users from login to purchase. Intuitively, the actions of users reveal their navigation patterns, decision-making processes, and, more crucially, intent. Fraudsters generally have a more specific purpose, so they don't spend too much time on searching. For example, Fig. 2 shows a fraudulent action sequence, in which the user spends 15 s to search an item, uses about 30 s to view the item (the jewelry and the price), and then spends only 10 s to checkout. In contrast, a benign

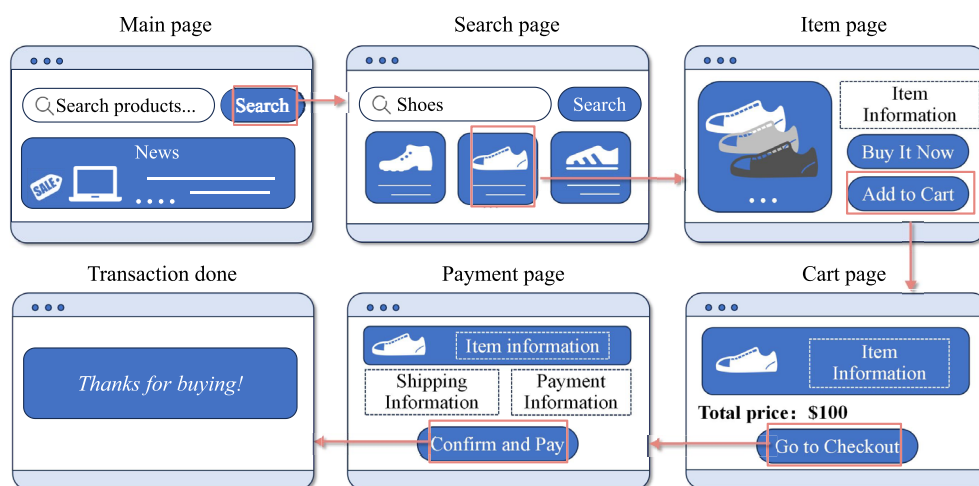


Fig. 1 Page view sequence of a typical shopping procedure on e-commerce platforms. It shows the usual steps a user takes from searching for a product to completing a purchase

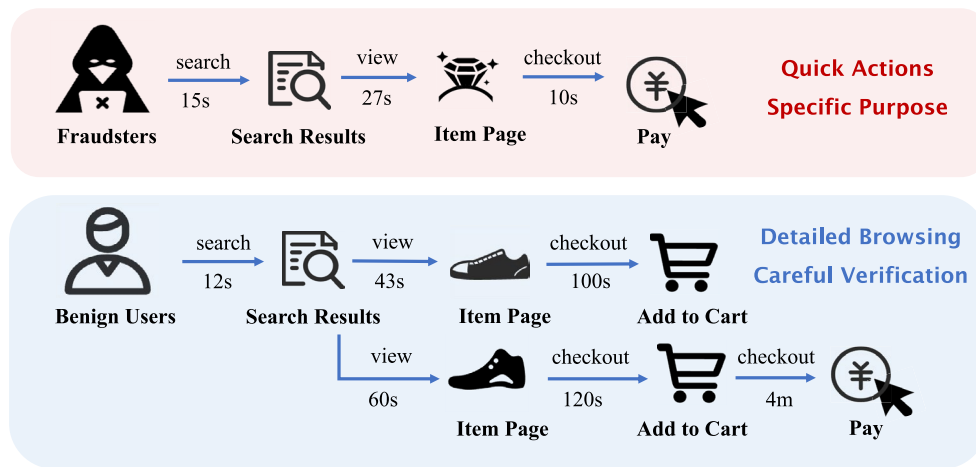


Fig. 2 Action sequences of fraudsters and benign users. Fraudsters' actions are goal-oriented, moving swiftly from search to purchase without detours while benign users prefer to take substantial time to select items and confirm details

user may spend more time viewing and selecting items, adding them to the cart, and carefully confirming details during checkout. Overall, we can observe that the fraudster has a clear intent and is more proficient in conducting actions, while the benign user does not have clear targets and therefore spends more time on browsing different products and carefully comparing prices before making the final decision. Fraudsters exhibit clear intent and efficiency, while benign users spend more time browsing and comparing products before deciding. This difference in behavior can help identify fraudulent activities.

Inspired by the distinct behaviors of users, it becomes popular to study user behaviors and offer perspectives on how users engage with a service or product. This can benefit various business units of e-commerce companies [16] in data analytics. User behavior represents a unique fingerprint that is difficult to forge, making it an effective indicator to distinguish between different users [17]. For example, in behavioral biology theory, continuous user authentication is perceived by analyzing the unique interaction between users and devices. Since user behavior describes the unique digital signature of the user and is hard to fabricate, user behavior analysis can provide knowledge to learn an individual's purpose and, therefore, to improve fraud detection capabilities.

Despite the previous efforts on fraud detection using user behavioral data, we observe several issues that have not been yet well studied. Despite the previous efforts on fraud detection using user behavioral data, we observe several issues that have not been yet well studied. **① How to build an end-to-end pipeline for identifying e-commerce frauds with user behavioral data in an industrial environment?** This is essential for practical e-commerce applications, but has not been studied. **② Can we leverage multiple user behavioral data sources?** The previous works [18–21] mostly focus on

inner-page behavioral data, typically mouse movement trajectory. However, other sources of user behavioral data are also valuable and have been yet ignored. For instance, since an entire user session on an e-commerce platform contains multiple pages, the inter-page viewing history can reveal user behavior and purpose. **③ How to properly preprocess user behavioral data for the downstream prediction model?** Some user behavioral data, e.g., mouse trajectory, cannot be directly fed into the detection model. For effectively preprocessing mouse trajectory, we need to concurrently consider the mouse positions, the temporal information during mouse movement, and diverse interfaces on user devices. **④ How to train a fraud detection model with multi-modal user behavioral data?** Given multiple sources of user behavioral data, the next issue is to construct a fraud detection model. This involves choosing the backbone model and the fusion method, which merges various inputs. **⑤ What can we learn from the prediction results?** Other than the model training, it is also interesting to analyze the predicted frauds. This can help the e-commerce platforms summarize typical fraudulent patterns, discover differences between fraudulent and benign activities, and provide explainability for the prediction results which is a vital concern of the risk control department.

To answer these questions, we conduct a thorough study of e-commerce fraud detection through user behavioral data. We design a fraud detection framework in a world-leading e-commerce company, Multi-Modal Behavioral Transformer (MMBT), which consists of an online engine and an offline engine. In addition to inner-page mouse trajectory, we include more relevant data from inter-page user behavior, e.g., the viewed page ID, the dwell time on each page, and the item information on each page. To adapt the mouse trajectory to the detection model, we abstract the mouse

trajectory as images, treat each mouse position as a pixel, and transform the mouse trajectory into a sequence of image patches. We adopt the Transformer model as the backbone to encode user behavioral data and leverage a multi-tower architecture to fuse different input sequences. Furthermore, we conduct a detailed case study w.r.t. the patterns of predictions, with which we draw insights for user behavior analysis in identifying e-commerce frauds. Our contributions are summarized as follows:

1. We design a fraud detection framework that leverages user behavioral data in a world-leading e-commerce company's production environment. Our framework has an online workflow for real-time inference and an offline workflow that trains the prediction model.
2. We propose using multiple sources of user behavioral data, including both inner-page mouse trajectory and inter-page viewing history, to identify e-commerce frauds in a unified framework.
3. To handle complex mouse movement trajectories across different user devices, we propose to treat the mouse trajectory within a page as an image, and each mouse hovers as a pixel. Then, we divide the image into small patches and transform the mouse trajectory into a patch sequence.
4. Adopting Transformer as the backbone, we construct a multi-tower architecture to learn user patterns by fusing multiple user behavioral data sources.
5. Extensive experiments on an industrial dataset in our company demonstrate that our proposed framework MMBT can effectively improve the performance of fraud detection compared to a range of baselines. To further verify the effectiveness of MMBT, we conduct a thorough empirical study on various sequence manipulation techniques and model design.

2 Related Work and Preliminaries

2.1 Fraud Detection

Fraud detection, aiming to identify and prevent fraudulent activities, is a crucial research topic in e-commerce. It focus on specific financial sectors such as credit cards [22, 23], securities [23, 24], financial statement [25, 26] and insurance [27, 28]. Most e-commerce platforms are implementing fraud detection algorithms to improve transaction security and reduce financial losses [16, 29–34]. Recurrent Neural Networks (RNN) are widely used to detect account hijacking and card faking events [30, 31] by modeling users' static profiles or dynamic click actions. Some researches construct relation graphs between users and transactions and develop Graph Neural Networks (GNN) for fraud

detection [35, 36]. Yin [16] proposes to construct a similarity soft-link graph and utilize GNN to aggregate similar node features. Zhang [29] employs a Competitive Graph Neural Network (CGNN) on the user-item graphs to distinct normal and fraudulent actions. Recently, due to the diversity of data collected by e-commerce platforms, multi-modal approaches are becoming the trend in fraud detection. Aras [37] proposes a multi-modal personal profiling system for identifying potential fraudulent airline ticket activities, which integrates multiple information sources such as credit card information, passenger name record information, and transaction records. Some other works [38, 39] use the Correspondence Learning (CL) scheme to learn relationships between multiple modalities explicitly. However, some user data are highly sensitive and not easy to collect [40]. How to maximize user privacy and security in fraud detection remains a significant challenge. In this study, our goal is to investigate a secure e-commerce fraud detection method using multiple data sources over a real-world dataset in the industry.

2.2 User Behavioral Data in E-Commerce

In the realm of e-commerce applications, particularly in the context of fraud detection, the intricacies of user interactions can be revealed through the lens of behavioral data [29]. Users' behavioral data are typically sourced from the logs of user actions in e-commerce digital interfaces, such as the website of an e-commerce platform or the application on PC and mobile devices. During the lifecycle of a user session, the e-commerce interfaces capture a spectrum of sequential user actions. Through an in-depth analysis of these user behavior data, we are able to draw deep insights into the interaction patterns and decision-making processes of users on e-commerce platforms. Unlike other methods, they preserve user privacy and allow identity verification or fraud detection without needing sensitive personal information or additional hardware [41]. There exist various sources of user behavior data. For example, mouse trajectory is very typical and can be easily collected, which directly reflects user actions in e-commerce activities. Researchers have developed various statistical and machine learning algorithms to leverage mouse dynamics for user identification and authentication. In some early works, Support Vector Machine (SVM) and K-Nearest Neighbors (KNN) are commonly used machine learning models [21, 42–47].

Recently, deep learning techniques are becoming more and more popular. Chong [48] conducts studies showing that 2D-CNN models outperform 1D-CNN and SVM models. They also explore user authentication using Long Short-Term Memory (LSTM) and hybrid CNN-LSTM networks [49]. Antal [50] proposes an improved 1D-CNN model using velocity as input instead of the raw coordinate.

However, the state-of-the-art 1D-CNN model focuses on velocity vectors, neglecting mouse position information, while the 2D-CNN model treats mouse trajectories as images, missing temporal details. Unlike previous works rely only on mouse movement trajectory, we extend our analysis beyond mouse dynamics to include other sources of user behavioral data. Our work introduces the viewing page sequence as a kind of behavioral biometrics, which can also play a role in fraud detection [16, 51]. Additionally, we systematically consider both the position information (overlooked by 1D-CNN) and temporal information (overlooked by 2D-CNN) of mouse dynamics.

2.3 Transformer-Based Behavior Analysis

Transformer was originally introduced in [52] and defined as an encoder-decoder architecture. Afterward, a trend emerged toward using Transformers for behavioral biometrics analysis. Stragapede [53] first proposes a keystroke verification system based on Transformer, designed for challenging free-text mobile scenarios. Havrylovych [54] integrates the Transformer attention module with different autoencoders. It proves that combining Transformer blocks with an undercomplete deterministic autoencoder yields the best performance in biometric verification and user authentication tasks. Recent Yoon [55] introduces a pre-trained Transformer model (PIEformer) capable of extracting discriminative embeddings within an open-set environment and achieving more secure and practical behavioral biometrics authentication.

Following previous works, we use Transformer model as backbone for feature extraction from user behavior data such as mouse movement sequences.

3 Methodology

In this section, we show how we identify e-commerce fraud through user behavior data. Note that our final solution does not emerge overnight; instead, we have studied a wide spectrum of design spaces, regarding data sources, data preprocessing, backbone model, and overall model architecture.

3.1 Overview of Fraud Detection Framework

Workflow Overview

Figure 3 shows the fraud detection pipeline with user behavior data. The flow of *online* processes is characterized by a continuous stream of behavioral event data. This includes, but is not limited to, data pertaining to page views and mouse trajectory patterns. The system is designed to ensure minimal latency, thereby enabling near-instantaneous acquisition of data after the occurrence of the event. Within the *offline* workflow, one can identify three principal components integral to the system's functionality. These components are designed for data storage, model training, and model evaluation. Their collective objective is to monitor model performance and facilitate iterative model refinement. This system incorporates Flink as the stream engine and runs different Transformer inputs in parallel. We list the *system modules* as follows.

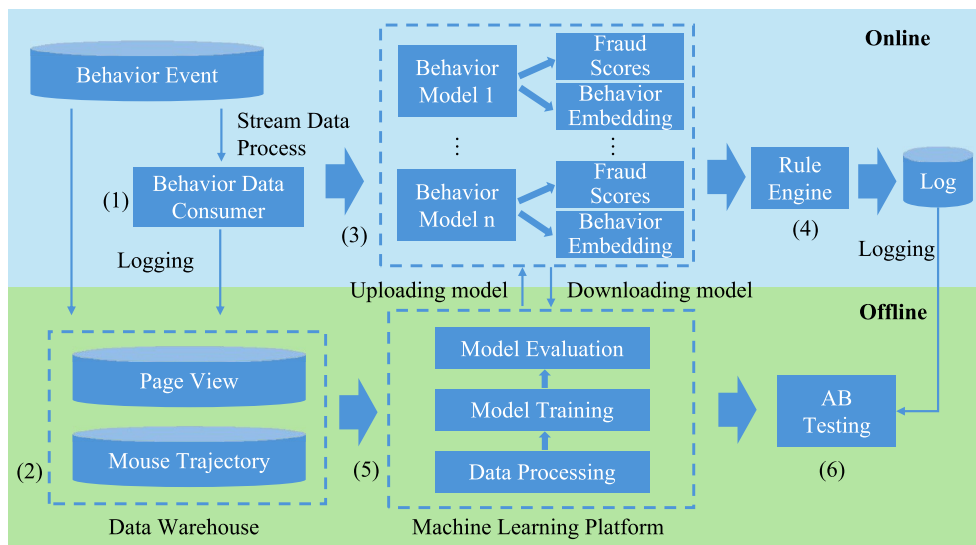


Fig. 3 Overview of fraud detection framework, Multi-Modal Behavioral Transformer (MMBT). It shows how data from page views and mouse trajectories are integrated to create behavior models for real-time fraud detection

1. *Data Consumer* The behavior data consumer functions as a data processing module, tasked with transforming incoming data into a format suitable for model ingestion. The online consumer has a relatively low latency, where the latency of the 99th percentile (P99) is < 50 ms.
2. *Data Warehouse* Both processed data and unaltered log data are archived within an offline data warehouse, facilitating subsequent analysis and predictive model training. In our use case, there are page view sequence and mouse trajectory data.
3. *Online Inference* These prepared inputs are then filtered into a variety of behavior models, each tailored to specific objectives. The outputs generated by these models encompass fraud scores and behavioral representations, the latter of which are termed behavioral embeddings. The inference part is the most expensive part of the whole inference pipeline; we have observed that the P99 latency for the online inference process is < 500 ms.
4. *Online Rule Engine* Model outputs (fraud scores and behavior embeddings) are fed into the rule engine that makes real-time decisions. Decisions and action results are stored into the data warehouse. From getting model scores to obtaining the final action, the P99 latency is < 50 ms.
5. *Offline Training* The machine learning platform serves as the nexus for data processing, model training, and subsequent evaluation. This platform is equipped with data processing modules that facilitate the reconfiguration of input data into new or required formats as necessary. Once the data is suitably prepared, the process of model training is initiated, followed by a thorough evaluation to ascertain performance improvements.
6. *AB Testing* Should the model demonstrate enhanced performance metrics, it is then progressed to the A/B testing module. Here, the model is subjected to an evaluation using live online traffic. If performance is satisfactory, the model file is updated online.

3.2 Behavior Data and Transaction Frauds

To build a fraud detection model, the first issue we need to solve is:

What data sources of user behavioral data are included as inputs for the fraud detection model?

In e-commerce scenarios, there are various kinds of behavior data sources. Considering the lifecycle of a user's session, the activity of users can be categorized into two aspects—text/inner-page actions and *inter-page* actions. Inspired as such, we try to learn user behavior from the perspective of both inner-page level and inter-page level.

- *Inner-page behavior data.* Similar to existing work [50], we choose the mouse movement trajectory as the main source of inner-page behavior data. The mouse trajectory includes the mouse position of each mouse hover, the specific time of each position, and the path of mouse movement inside a page.
- *Inter-page behavior data.* Intuitively, the chronological sequence of pages during a session can reveal a user's pattern, decision-making process, and intention. We collect all pages viewed during a buyer's digital session, from which we further extract the corresponding dwell time and item information.

The above two types of user behavior data provide abundant biometric information. By collaboratively learning from these data sources, we can identify if a transaction is fraudulent or not. Therefore, fraud detection is actually a *binary classification task*. We mainly consider the following two types of actions as fraud: account takeover (ATO) and stolen financial (SF).

- *Account takeover* In an e-commerce platform refers to the unauthorized access and control of a user's account by a third party. This can happen when a hacker illegally gains access to a user's login credentials, such as username and password, and uses them to log in to the victim user's account.
- *Stolen financial* Refers to unauthorized access and use of a user's financial data, such as credit card information, bank account details, or other sensitive financial information, by a third party.

These two types of e-commerce fraud can lead to serious losses to targeted users. To avoid such situations, it is hence the responsibility of e-commerce platforms to identify fraudulent transactions and protect consumer rights.

3.3 Data Preprocessing

The original user behavior data cannot be used directly. Instead, it needs proper data preprocessing before being fed into the fraud detection model. We work with two types of data in various modalities, mouse trajectory (inner-page) and page viewing history (inter-page), containing four series of sequential data — patch index sequence, page time sequence, page ID sequence, and item price sequence.

Mouse Trajectory Data Raw mouse trajectory data includes position coordinates and timestamps for each mouse point. We need to transform the trajectory to sequence data. The most *straightforward* way is to obtain the absolute position of each mouse hover and assemble them into a sequence according to their chronological order. A similar strategy proposed in a prior work [50] is to use mouse movement

velocity instead of the absolute position. However, users use different devices with diverse screen sizes and resolutions. For example, the screen of a mobile device is usually a portrait, but that of a computer is often landscaped; the size of a computer monitor can vary from 15 inches to more than 30 inches, and the resolution can range from 1 to 10 K. For the same user behavior on different devices, using absolute position or movement velocity may produce distinct sequences. To tackle the multi-device issue, we ask:

Instead of transforming the mouse trajectory in a fixed manner, can we do it dynamically using the relative position of mouse movement?

To alleviate the impact of screen size and resolution, a possible strategy is to divide the screen into the same grid and use the grid position. Inspired by this, we treat the mouse trajectory as an image in which each position of the mouse movement can be represented as a pixel. We divide the image into $M \times N$ patches, where M is the number of patches along the x axis and N is along the y axis. The minimum and maximum x coordinates of the screen are denoted by $x_{\min}$ and $x_{\max}$, and the minimum and maximum y coordinates are $y_{\min}$ and $y_{\max}$. Hence, the length of each patch along the x -axis is $\Delta x = \frac{x_{\max} - x_{\min}}{M}$; likewise, the patch length along the y -axis is $\Delta y = \frac{y_{\max} - y_{\min}}{N}$. Then, we calculate the patch index i of a mouse position (x, y) as:

$$i = \left\lfloor \frac{y - y_{\min}}{\Delta y} \right\rfloor \times M + \left\lfloor \frac{x - x_{\min}}{\Delta x} \right\rfloor \quad (1)$$

where $i_x = \left\lfloor \frac{x - x_{\min}}{\Delta x} \right\rfloor$ is the x -axis index of the patch and $i_y = \left\lfloor \frac{y - y_{\min}}{\Delta y} \right\rfloor$ is the y -axis index of the patch.

This method converts any point coordinate (x, y) in the image to a unique patch index that does not correlate with screen size and resolution. It treats all screens as having a similar patch layout, meaning that similar user behaviors (such as clicking a button at the same relative position) are

transformed into the same patch index sequence regardless of screen size. We show an example in Fig. 4 to illustrate this data preprocessing method. On the checkout page, users confirm their shipping address, check product information, fill in payment details, and click “Confirm and Pay”, generating a series of mouse trajectory points. Our method uses 30×30 patches, converting mouse positions to patch index sequences. In the zoom-in image, point (529, 321) is located within patch 502, so we use patch index value 502 to replace its original coordinates. Similarly, 621 is used to represent the coordinate point (501, 216). Consequently, we transform the original mouse trajectory into a 1-D patch index sequence arranged in chronological order [416, 502, 621, 736]. Our patching method standardizes screen layouts, ensuring that similar behaviors produce similar patch sequences. Note that a button may span multiple patches. But since these patches are adjacent, the indices of these patches are close. The model can also implicitly learn that some patches belong to the same operation.

Page ID Sequence To generate the page ID sequence (e.g., “search”-“view”-“checkout” in Fig. 2), we first examine the frequency distribution of page IDs and remove those that occur below a certain threshold. We use the entire dataset to calculate the frequency. After filtering, we re-encode the remaining page IDs by mapping each unique page ID to a unique integer sequentially starting from 0.

Item Price Sequence For item price data, we make bins to the original price. First, we analyze the distribution of item prices in the whole dataset and create 1500 bins. We choose equal-frequency binning as our binning strategy [56]. After binning, each price is replaced by its corresponding bin index. These indices should start from 0 and go up to (number of bins-1).

3.4 Model Design

After preprocessing the original user behavior data, we then study the design of the fraud detection model. We first

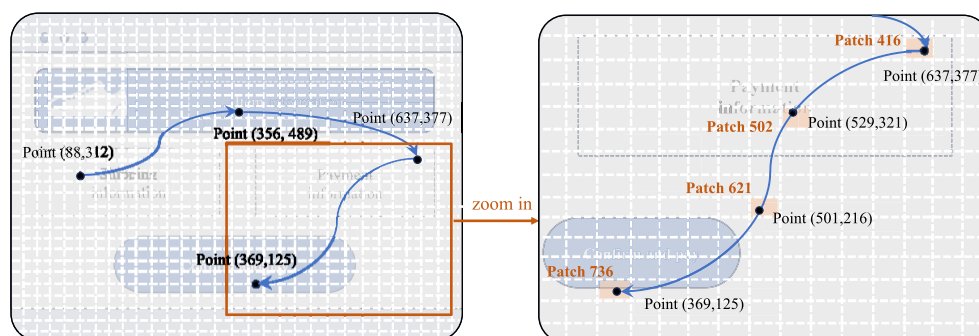


Fig. 4 Patching of mouse trajectory. We divide the screen into $M \times N$ patches and converting mouse positions to patch index sequences. For example, point (529,321) is located within patch 502, so we use patch index value 502 to replace its original coordinates

discuss existing CNN-based methods (which we use as experimental baselines in Sect. 4.1) and then introduce our proposed method using the Transformer model as backbone.

3.4.1 CNN Backbone Model

Previous works [48–50] normally choose CNN as the backbone model to learn the knowledge from the mouse trajectory.

2D-CNN Model Chong [49] propose to apply a 2D-CNN model for mouse trajectories. As shown in Fig. 5a, the model employs a GoogLeNet architecture [57] with convolutional layers and inception modules, and outputs classification probabilities through a linear layer and sigmoid function. The raw coordinates are segmented by time differences, starting a new mouse movement sequence when the intervals exceed one second. “Mouse Trajectory Img” are images visualized from these mouse movement sequences and then resized to 448×448 . The model processes the images through convolutional layers (7×7 , 1×1 , 3×3) for feature extraction and inception modules to capture multi-scale features, followed by global average pooling before producing the final output. Although effective in capturing movement patterns, the 2D-CNN model does not include

temporal information, as well as the ability to account for the speed and direction of mouse movements.

1D-CNN Model To address the limitation of the 2D-CNN backbone, researchers tend to convert mouse trajectory data to one-dimensional sequences and feed them into a 1D-CNN model. We discuss two variants of 1D-CNN here, Velocity-1D-CNN [50] and Coord-1D-CNN [49].

As shown in Fig. 5b, the Velocity-1D-CNN model [50] inputs velocity data calculated from raw mouse coordinates, processes them through parallel convolutional layers with pooling and dropout, then outputs probabilities via a linear layer and sigmoid activation. The input is divided into two towers, each passing through convolutional layers with kernel size 10 (Conv1 and Conv2) and 5 (Conv4 and Conv5), followed by global max pooling and concatenation. Note that a tower model with different kernel sizes was used, which helped the network to learn input sequences on different time scales.

The structure of Coord-1D-CNN [49] is similar to Velocity-1D-CNN, but Coord-1D-CNN employs a convolutional layer with shared weights in the second convolutional layer. Additionally, Coord-1D-CNN uses normalized raw mouse trajectory coordinates as input, while Velocity-1D-CNN uses velocity values.

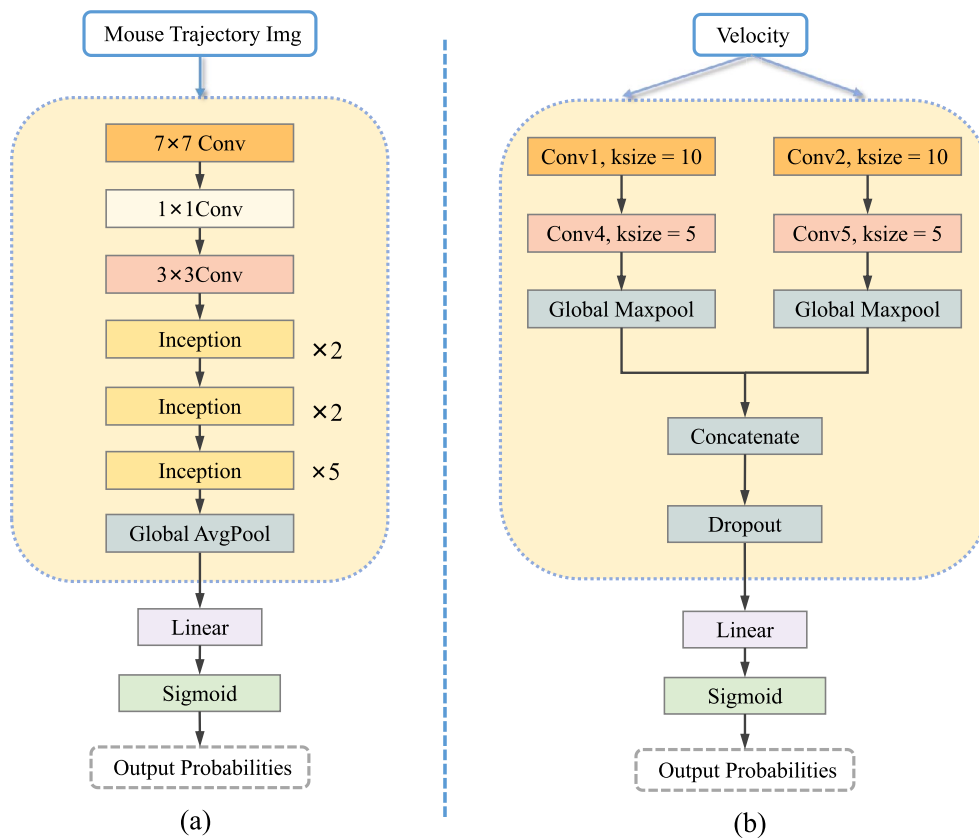


Fig. 5 CNN-based model architecture: **a** 2D-CNN model, **b** 1D-CNN model

For example, consider a mouse point $P(x, y, t)$, where x and y are the coordinates of the point, and t is the timestamp at the current moment. Coord-1D-CNN inputs the normalized coordinates (x', y') , with values ranging from $[0, 1]$ into the model. In contrast, for Velocity-1D-CNN, it calculates the dwell time Δt and the absolute displacement values along the x and y axes, Δx and Δy , between every two points. The velocity at each point $\left(\frac{\Delta x}{\Delta t}, \frac{\Delta y}{\Delta t}\right)$ is then used as input for Velocity-1D-CNN.

Although the 1D-CNN models maintain the chronological order of mouse trajectories, they struggle to learn the implicit relationships between mouse positions effectively due to a limited receptive field. Since mouse trajectories are often long and disordered, understanding correlations between different trajectory segments can provide a better insights into user behavior and intent.

3.4.2 Transformer Backbone Model

To address CNN limitations, we use Transformer as the backbone for its superior ability to capture long-term dependencies and complex global patterns. To better understand the influences of multiple data sources, we construct both single-modal architecture and multi-modal architecture, as shown in Fig. 6.

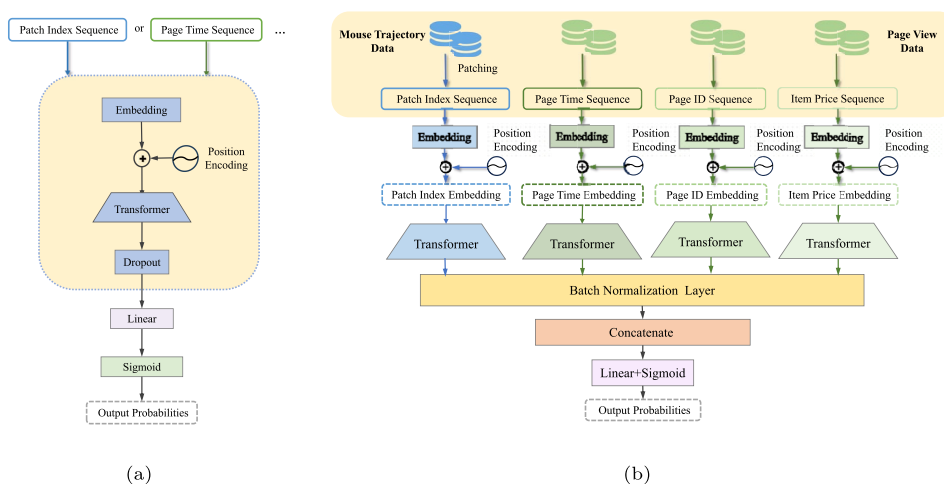
Single-modal Transformer Architecture The input is either patch index sequence, page time sequence, page ID sequence, or item price sequence. We use the classic Transformer Encoder structure, which establishes multiple layers. These sequences are processed using the methods we discussed in the data preprocessing (Sect. 3.3), then transformed into high-dimensional representations through an embedding layer with positional encoding. Next, these embedding vectors are processed through Transformer Encoders with multi-head attention to capture contextual relationships. Finally, the features of the first token

(token0) are fed into a linear layer to output fraud probabilities via a sigmoid function.

Multi-modal Transformer Architecture To fuse multiple sources of behavioral data, we establish a multi-modal architecture called MMBT (Multi-Modal Behavioral Transformer). Data from mouse trajectory and page viewing history provide four sequential inputs—patch index sequence, page time sequence, page ID sequence, and item price sequence. Each sequence is processed by independent Transformer encoders to extract user behavior features. After the fusion layer, we use these fused features to predict the probability of fraud. Below, we introduce the four major components of the model structure.

1. **Multiple Transformer Encoders** We use four independent Transformer encoders, each processing a distinct input sequence. These encoders do not share parameters, allowing each to specialize and effectively capture unique patterns in the corresponding user behavioral data.
2. **Batch Normalization Layer** After being processed by Transformer Encoder, each group of feature representations is fed into a shared-parameter batch normalization (BatchNorm) layer to align the features.
3. **Feature Fusion** After the BatchNorm layer, the four groups of feature representations are merged into a single high-dimensional feature vector through a *concatenate* operation. This fusion of multi-modal features allows the model to utilize various aspects of user behavior data for predictions.
4. **Classification Model and Output** The fused feature vector is fed into a Multilayer Perceptron (MLP) with multiple fully connected layers and non-linear activations. The model outputs prediction probabilities via a sigmoid function, restricting the values to the $[0, 1]$ range for classification tasks.

Fig. 6 Transformer-based model architecture: **a** Single-modal transformer model, **b** Multi-modal transformer model



4 Evaluation

In this section, we conduct extensive experiments to evaluate the performance of our proposed approach. We first compare our method with several state-of-the-art models on a real-world dataset from the company platform. Furthermore, we conduct a detailed empirical anatomy to study the design space containing various data processing techniques and underlying models. In this way, we validate the effectiveness of each component in our proposed framework. Specifically, our goal is to answer the following questions we have asked in the introduction.

- (1) RQ1: Can our method improve the overall performance of fraud detection? (Sect. 4.2)
- (2) RQ2: Can the multi-modal Transformer model outperform the single-modal CNN-based and Transformer-based counterparts? (Sect. 4.3)
- (3) RQ3: What are the benefits of behavioral data other than the mouse trajectory in fraud detection? (Sect. 4.3)
- (4) RQ4: How to effectively fuse the representations of different types of user behavioral sequences? (Sect. 4.4)
- (5) RQ5: What can we learn from predicted results? (Sects. 4.4 and 4.5)

4.1 Experimental Setup

Dataset We conduct our experiments on a real-world dataset in a world-leading e-commerce company which contains more than 3 million real transactional records from diverse devices. The frauds are labeled based on actual chargeback events, such as credit card fraud reported by users. Fraud events constitute 5% of the entire dataset.

Each instance in the dataset contains a transaction ID, mouse trajectory data, page viewing sequences, user engagement duration, and product price information. By analyzing browsing behaviors, we aim to categorize users into groups based on distinct behavioral characteristics and assess transaction and user riskiness, as illustrated in Fig. 3. The dataset is randomly split into training, validation, and test sets in an 8:1:1 ratio. Unless otherwise stated, performance metrics are reported on the test set.

Baselines We compare our proposed method MMBT with four baseline methods. These methods use CNN as the backbone, taking mouse trajectory data as input but using different model structures and data processing methods.

In addition, we propose a new baseline Vel-Del-1D-CNN, which is a variant of Velocity-1D-CNN. For better understanding whether embracing more data sources without delicate data preprocessing is beneficial, we add the

offset of mouse movement as its input. Each input item is a five-dimensional vector $\left(\frac{dx}{dt}, \frac{dy}{dt}, \Delta x, \Delta y, \Delta t\right)$, in which Δx is the offset along the x -axis, Δy is that along the y -axis, and Δt is the dwell time.

Evaluation Metrics We use APR (Average Precision-Recall), AUC (Area Under the Curve) and precision to measure the performance of our framework and the baselines in detecting fraud transactions [58]. APR, the average of precision and recall, provides a global perspective on the model's ability to distinguish between fraudulent and normal transactions. AUC quantifies the model's ability to distinguish between different classes. Specifically, in practical e-commerce applications, balancing sensitivity and specificity in classifying fraudulent and normal transactions is crucial. Company's risk control department often uses prediction precision at a specific recall, which means, for a given recall threshold, how much precision the model can provide. Thus, in addition to APR and AUC, we report the precision at a fixed recall of 10%.

Implementations For training the Transformer model, we set the dimension of the embedding vectors as 768 and use 6 Transformer layers, each with 8 attention heads. We apply a dropout probability of 0.5 after the last layer. All the methods are implemented in Pytorch, and we use Adam [59] as the optimization algorithm. For Adam, we set the learning rate to $1e - 4$. The batch size is set to 32. We conduct experiments on 2 GPUs with 32 G memory.

4.2 Performance Comparison on Fraud Detection

We compare MMBT with the baseline methods, w.r.t. APR, AUC, and precision with a recall threshold of 0.1.

Model Performance The overall performance of the model is shown in Table 1. The first four rows show the results of Coord-1D-CNN, Velocity-1D-CNN, Vel-Del-1D-CNN, and 2D-CNN with our method MMBT presented in the last row. MMBT achieves the best results across all metrics, followed by Velocity-1D-CNN and 2D-CNN. Specifically, the APR of MMBT is 66.09%, which is 1.33%

Table 1 Comparison with the baselines

Methods	APR	AUC	Precision@ Recall = 0.1
Coord-1D-CNN	0.5292	0.5668	0.5239
Velocity-1D-CNN	0.6476	0.6466	0.8194
Vel-Del-1D-CNN	0.5121	0.5161	0.5653
2D-CNN	0.6340	0.6472	0.7654
MMBT (Ours)	0.6609	0.6760	0.8889

The best results are highlighted in bold

higher than that of the second-best model Velocity-1D-CNN. Improvements on AUC and Precision@Recall=0.1 are 2.94% and 6.95%, respectively. This demonstrates that incorporating page viewing information and proper preprocessing enhances performance. Vel-Del-1D-CNN shows a significant performance drop compared to Velocity-1D-CNN due to added displacement and dwell time without proper preprocessing, introducing noise.

System Efficiency The primary modules that influence the efficiency of the system are as follows, each contributing to its overall efficiency:

- *Data Consumer* efficiently transforms the incoming behavioral data for the downstream model with a P99 latency < 50 ms.
- *Data Warehouse* archives both processed and raw data, supporting analysis and model training. Hourly updates are conducted to ensure the efficiency of analysis and utilization.
- *Online Inference* processes data through behavior models to generate outputs like fraud scores, with a P99 latency < 500ms, despite being the most resource-intensive online step.
- *Online Rule Engine* implements real-time decisions based on model output with a P99 latency < 50 ms.
- *Offline Training* takes roughly two days on our dataset, using 2 GPUs. It is performed periodically offline and therefore does not affect the online process.

In general, the system can efficiently manage data processing, storage, and real-time analysis, maintaining high speed and performance across its pipeline.

4.3 Anatomy of Multi-modal Input Sequences

MMBT uses various user behavioral data for fraud detection, including four sequences extracted from mouse trajectory and page view data—patch index sequence, page time sequence, page ID sequence, and item price sequence. This section presents an empirical study of these input sequences.

Masking One Input Sequence To measure the impact of each input sequence, we individually mask one of these inputs and train a model with the remaining three sequences. Table 2 shows the evaluation metrics. All four input sequences improve the model performance. The patch index sequence has the greatest impact, resulting in a 21% increase in APR and a 19% increase in AUC. The improvements brought by the input sequences from the page viewing data are less significant. Specifically, the item price sequence increases APR by 1.86% and AUC by 1.4%, while page time and page ID improve APR by 1.44% and 0.4%, and AUC by 0.6% and 0.1%, respectively. These results suggest product price information is particularly useful for fraud detection, as it correlates with transaction value and fraudsters often target expensive items. In contrast, page time and page ID sequences, while indicative of user interest, are less predictive overall.

Masking Two Input Sequences In the previous experiments, we prove that the page ID sequence has the least effect on the model efficiency. To optimize sequence combinations for better efficiency and reduced redundancy without sacrificing accuracy, we explore two-input experiments after removing the page ID. Table 3 shows that removing the page time sequence results in a slight decrease in APR and AUC, but a 2.54% increase in precision. Masking the item price

Table 2 Study of input sequences (masking one input sequence)

Row	Patch index	Page time	Page ID	Item price	APR	AUC	Precision@ Recall = 0.1
(1)	✓	✓	✓	✓	0.6609	0.6760	0.8889
(2)		✓	✓	✓	0.4507	0.4904	0.5099
(3)	✓		✓	✓	0.6465	0.6700	0.8502
(4)	✓	✓		✓	0.6567	0.6751	0.8236
(5)	✓	✓	✓		0.6423	0.6620	0.8502

The best results are highlighted in bold

Table 3 Study of input sequences (masking two input sequences)

Row	Patch index	Page time	Page ID	Item price	APR	AUC	Precision@ Recall = 0.1
(1)	✓	✓		✓	0.6567	0.6751	0.8236
(2)		✓		✓	0.4627	0.5000	0.4524
(3)	✓			✓	0.6511	0.6701	0.8756
(4)	✓	✓			0.6526	0.6723	0.8670

The best results are highlighted in bold

sequence (Table 3, Row 5) leads to a more noticeable performance drop than masking the page time sequence (Row 3) or page ID sequence (Row 4). When the page ID sequence is removed (Table 3), removing the item price sequence (Row 4) has a smaller impact than removing the page time sequence (Row 3). This suggests that page IDs and item prices may be related, with item prices potentially providing overlapping information with page IDs.

If feature engineering time is limited, removing the page ID and item price sequences can reduce redundancy and maintain crucial features for efficient fraud detection, as shown in Table 3 (Row 4). This approach simplifies the model while preserving robust APR and AUC with minimal performance reduction. Removing both the page time and page ID sequences (Table 3, Row 3) results in lower APR and AUC values than removing only the page ID (Row 1), but still retains relatively high performance, indicating these sequences may provide overlapping information.

Single-Modal Versus Multi-modal In Sect. 3.4.2, we discuss the design of both single-modal and multi-modal model architectures. Next, we study the effect of multi-modal mechanisms by independently evaluating each input sequence with the single-modal model under the same experimental configuration. From the experimental results in Table 4, we can observe that the multi-modal model using all four sequences outperforms the single-modal models using only one sequence. Among the single-modal models, the one using the patch index sequence achieves the best results, with an APR of 63.48% and an AUC of 65.34%. Adding supplementary page viewing inputs further improves performance. Among the page viewing data, the item price sequence performs best in a single-modal Transformer, highlighting its significance in depicting fraud patterns. In contrast, the page time sequence and page ID sequence perform worse when used alone.

4.4 Anatomy of Fusion Methods

The multi-modal architecture fuses multiple data sources using either early or late fusion methods. Early-fusion

combines Transformer output embedding vectors using concatenation, attention, average, or weighted sum. Late-fusion merges output probabilities of single-modal models using max, average, or weighted sum. For example, in Fig. 6b, early fusion combines the output embeddings from different data sources after the Transformer encoders. In contrast, late fusion merges the final output probabilities from individual models. We conduct a detailed study exploring these fusion methods for both types.

Early-Fusion Methods Let $\mathbf{v}_{\text{patch}}$, $\mathbf{v}_{\text{time}}$, $\mathbf{v}_{\text{id}}$, and $\mathbf{v}_{\text{price}}$ denote the embedding vectors of patch index, page time, page ID, and item price respectively. Let F be the fusion function, then the early-fusion yields:

$$\mathbf{v}_{\text{fusion}} = F(\mathbf{v}_{\text{patch}}, \mathbf{v}_{\text{time}}, \mathbf{v}_{\text{id}}, \mathbf{v}_{\text{price}}), \quad (2)$$

which is then fed to fully-connected layers, as illustrated in Fig. 6b. We consider the following candidate fusion functions F :

- *Concatenation*: All vectors are concatenated into a single vector:

$$\mathbf{v}_{\text{fusion}} = [\mathbf{v}_{\text{patch}} \parallel \mathbf{v}_{\text{time}} \parallel \mathbf{v}_{\text{id}} \parallel \mathbf{v}_{\text{price}}] \quad (3)$$

where $\parallel$ denotes vector concatenation operation.

- *Attention*: The attention strategy is widely adopted to learn correlations among items within a sequence. This mechanism contains the following steps:

1. *Define Query, Key, and Value* Let $\mathbf{Q} = \mathbf{v}_{\text{patch}}$ be the query vector, and $\mathbf{K} = \mathbf{V} = [\mathbf{v}_{\text{time}}; \mathbf{v}_{\text{id}}; \mathbf{v}_{\text{price}}]$, where all vectors generated from the page viewing data are stacked to form the key and value matrices.
2. *Compute Attention Scores* The attention scores are computed as the dot product of the query with all keys, followed by a softmax operation to obtain the weights:

$$\mathbf{A} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right) \quad (4)$$

where d_k is the dimensionality of the key vectors helps to stabilize the gradients. In this case, since $\mathbf{Q}$ is a single vector and $\mathbf{K}$ is a matrix, $\mathbf{Q}\mathbf{K}^T$ results in a vector of scores, and $\mathbf{A}$ is the vector of attention weights after applying softmax.

3. *Apply Attention to Values* The output vector, which is the fused feature vector, is obtained by weighting the values $\mathbf{V}$ with the attention scores $\mathbf{A}$

$$\mathbf{v}_{\text{fusion}} = \mathbf{A}\mathbf{V}. \quad (5)$$

Here, $\mathbf{v}_{\text{fusion}}$ is the contextually enriched representation that combines information from the four input

Table 4 Comparison of single-modal and multi-modal architectures

Input	APR	AUC	Precision@ Recall = 0.1
Patch index	<u>0.6348</u>	<u>0.6534</u>	<u>0.7759</u>
Page time	0.4588	0.4920	0.4453
Page ID	0.4650	0.5034	0.4667
Item price	0.4738	0.5139	0.4656
MMBT (with all above)	0.6609	0.6760	0.8889

The best results are highlighted in bold, and the second-best results are highlighted in underline

vectors according to their relevance to the query vector. This attention-based approach enables the model to focus on the most relevant features while reducing the influence of less important ones.

- *Avg*: Take the entry-wise average of the four vectors:

$$\mathbf{v}_{\text{fusion}} = \frac{\mathbf{v}_{\text{patch}} + \mathbf{v}_{\text{time}} + \mathbf{v}_{\text{id}} + \mathbf{v}_{\text{price}}}{4}. \quad (6)$$

- *Weighted Sum*: For a given weight α , compute the weighted sum of four vectors:

$$\mathbf{v}_{\text{fusion}} = \frac{\alpha}{3}(\mathbf{v}_{\text{time}} + \mathbf{v}_{\text{id}} + \mathbf{v}_{\text{price}}) + (1 - \alpha)\mathbf{v}_{\text{patch}} \quad (7)$$

where the weight α is set to 0.2 by delicate tuning.

Late-fusion Methods Likewise, let $\mathbf{p}_{\text{patch}}$, $\mathbf{p}_{\text{time}}$, $\mathbf{p}_{\text{id}}$, $\mathbf{p}_{\text{price}}$ denote the output probabilities of the corresponding single-modal models. Then late-fusion yields:

$$\mathbf{p}_{\text{fusion}} = F(\mathbf{p}_{\text{patch}}, \mathbf{p}_{\text{time}}, \mathbf{p}_{\text{id}}, \mathbf{p}_{\text{price}}), \quad (8)$$

which directly serves as the final probability.

- *Max*: Take the entry-wise maximum of the four predicted probabilities:

$$\mathbf{p}_{\text{fusion}} = \max(\mathbf{p}_{\text{patch}}, \mathbf{p}_{\text{time}}, \mathbf{p}_{\text{id}}, \mathbf{p}_{\text{price}}). \quad (9)$$

- *Avg*: Take the entry-wise average of the four predicted probabilities:

$$\mathbf{p}_{\text{fusion}} = \frac{\mathbf{p}_{\text{patch}} + \mathbf{p}_{\text{time}} + \mathbf{p}_{\text{id}} + \mathbf{p}_{\text{price}}}{4}. \quad (10)$$

- *Weighted Sum*: For a given weight α , compute the weighted sum of four predicted probabilities:

$$\mathbf{p}_{\text{fusion}} = \frac{\alpha}{3}(\mathbf{p}_{\text{time}} + \mathbf{p}_{\text{id}} + \mathbf{p}_{\text{price}}) + (1 - \alpha)\mathbf{p}_{\text{patch}} \quad (11)$$

where the weight α is set to be the same as the early-fusion.

Results and Analysis The detailed results are presented in Table 5. Among various data fusion strategies, the average strategy for early-fusion achieves the highest APR and AUC. The concatenation method ranks second in APR and AUC but achieves the highest precision by maintaining the integrity of information from various data sources. Using the attention mechanism for early-fusion does not perform well, as the patch index sequence dominates overall model performance, and the attention-based method struggles to capture complex interactions between data sources with significant differences. Overall, early-fusion methods outperform late-fusion methods, suggesting that early-fusion better mines potential correlations and complementarities between data

Table 5 Study of different fusion methods

Fusion methods		APR	AUC	Precision@ Recall=0.1
Early-fusion	Concat	<u>0.6609</u>	<u>0.6760</u>	0.8889
	Attention	0.4664	0.5051	0.4583
	Avg	0.6610	0.6806	0.8627
	Weighted sum	0.6413	0.6586	0.8585
Late-fusion	Max	0.4893	0.5124	0.5714
	Avg	0.6338	0.6739	0.7489
	Weighted sum	0.6472	0.6659	<u>0.8713</u>

The best results are highlighted in bold and the second-best results are highlighted in underline

sources, thereby integrating information more efficiently and enhancing model recognition capability. In contrast, late-fusion may be constrained by the loss of single model outputs, limiting the ability to fully leverage complementary information from different sources.

4.5 Case Studies

In this section, we provide concrete examples and aggregate statistics to illustrate the inner-page and inter-page differences between fraudulent and benign users. We first delve into the mouse trajectory, and then discuss the summary statistics on inter-page page viewing sequences.

Inner Page: Visualization of Mouse Trajectory To illustrate user behavior, we traced mouse trajectories of a fraudster and an ordinary user on a checkout page, as shown in Fig. 7a and b. Different colors represent time changes. Comparing these two figures, we have the following observations: 1) The fraudster's mouse movements are purposeful and direct, with few repeated paths, while the ordinary user's trajectory is complex with many hovers, turns, and repeated paths. 2) The ordinary user spends over 300 s, significantly longer than the fraudster's 130 s. These differences may arise from unfamiliarity with the platform, lack of a direct purpose, hesitation, or interruptions.

Inter Page: Comparison of Page Viewing Sequences In addition to the inner-page mouse trajectory, we also conduct an anatomy of the inter-page page viewing sequence. We extract the page viewing path of a user and the corresponding dwell time from transaction logs that record user history from the login to the user checkout and pay. We collect thousands of examples from the logs of company's e-commerce application and summarize the results. Similar to our observations in the previous anatomy of mouse trajectory, we find that fraudsters are more purposeful, more proficient in their operations, spend less time, and browse fewer pages. Overall, compared with normal transaction (Non-ATO) and account takeover (ATO) transactions, the Non-ATO users'

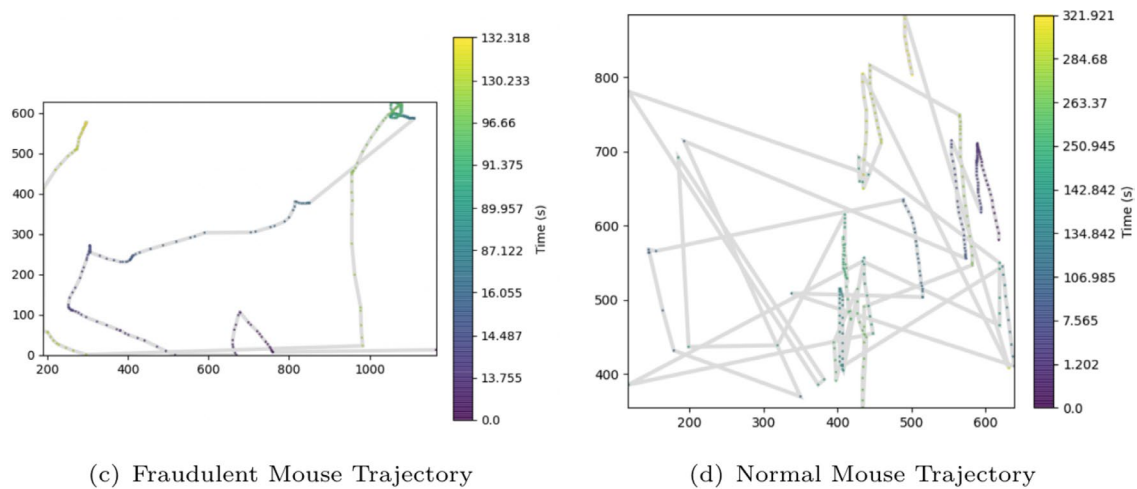


Fig. 7 Examples of fraudulent mouse trajectory and normal mouse trajectory

average and medium viewed page counts are almost 1.5 × more than the ATO users. The Non-ATO users' average dwell time, from user login session to checkout, is roughly one minute longer than that of ATO users.

5 Conclusion and Future Work

We are the first in the industry to fully utilize various behavioral data for e-commerce fraud detection. Our approach includes selecting data sources, preprocessing behavioral data, training deep learning models, and evaluating results.

We choose Transformer, the current mainstream deep learning network, as the backbone and propose a multi-modal approach to fuse both inter-page and inner-page sequences. Compared to the state-of-the-art approaches, our proposed framework MMBT obtains significant performance improvements by up to 7% on precision@recall = 0.1, which is an evaluation metric commonly used in the business case. We also conduct an anatomy regarding the input sequences and fusion methods. In the future, we plan to leverage additional page details in various scenarios. While current fraud detection focuses on the purchase stage, significant fraud also occurs during registration and login. These fraudsters also have unique behavioral patterns, so it is interesting to take advantage of behavioral data to identify fraudulent patterns and reduce losses.

Acknowledgements This work was sponsored by Key R&D Program of Hubei Province (2023BAB077, 2023BAB170), National Natural Science Foundation of China (62472327), and the Fundamental Research Funds for the Central Universities (2042023kf0219).

Author Contributions The first and second author were primarily responsible for writing the methodology and the code of the paper, and the other authors were mainly responsible for the other sections.

Funding This work was sponsored by Key R&D Program of Hubei Province (2023BAB077, 2023BAB170), National Natural Science Foundation of China (62472327), and the Fundamental Research Funds for the Central Universities (2042023kf0219).

Data Availability The data and code used in this study are not available due to the company privacy rules. Other materials are available upon request.

Declarations

Conflict of interest The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Bhattacharyya S, Jha S, Tharakunnel K, Westland JC (2011) Data mining for credit card fraud: a comparative study. *Decis Support Syst* 50(3):602–613
2. Hilal W, Gadsden SA, Yawney J (2022) Financial fraud: a review of anomaly detection techniques and recent advances. *Expert Syst Appl* 193:116429
3. Breuer A, Eilat R, Weinsberg U (2020) Friend or faux: graph-based early detection of fake accounts on social networks. In: *Proceedings of the web conference*, pp 1287–1297
4. Wu J, Yao M, Wu D, Chi M, Wang B, Wu R, Fu X, Meng C, Wang (2023) Dedgat: dual embedding of directed graph attention

- networks for detecting financial risk. arXiv preprint [arXiv:2303.03933](https://arxiv.org/abs/2303.03933)
5. Li S, Yang J, Liang G, Li T, Zhao K (2022) Sybilflyover: heterogeneous graph-based fake account detection model on social networks. *Knowl-Based Syst* 258:110038
 6. Chai Z, Yang Y, Dan J, Tian S, Meng C, Wang W, Sun Y (2023) Towards learning to discover money laundering sub-network in massive transaction network. In: *Proceedings of the AAAI conference on artificial intelligence*, vol 37, No 12, pp 14153–14160
 7. Li X, Li Y, Mo X, Xiao H, Shen Y, Chen L (2023) Diga: guided diffusion model for graph recovery in anti-money laundering. In: *Proceedings of the ACM SIGKDD conference on knowledge discovery and data mining*, pp 4404–4413
 8. Wang J, Wen R, Wu C, Huang Y, Xiong J (2019) Fdgars: fraudster detection via graph convolutional networks in online app review system. In: *Companion proceedings of the world wide web conference*, pp 310–316
 9. Shu K, Mahudeswaran D, Wang S, Liu H (2020) Hierarchical propagation networks for fake news detection: investigation and exploitation. In: *Proceedings of the international AAAI conference on web and social media*, vol 14, No 1, pp 626–637
 10. Cao B, Mao M, Viidu S, Philip SY (2017) Hitfraud: a broad learning approach for collective fraud detection in heterogeneous information networks. In: *IEEE international conference on data mining*, pp 769–774
 11. Zhu Y, Xi D, Song B, Zhuang F, Chen S, Gu X, He Q (2020) Modeling user's behavior sequences with hierarchical explainable network for cross-domain fraud detection. In: *Proceedings of the web conference*, pp 928–938
 12. Cao S, Yang X, Chen C, Zhou J, Li X, Qi Y (2019) Titant: online real-time transaction fraud detection in ant financial. arXiv preprint [arXiv:1906.07407](https://arxiv.org/abs/1906.07407)
 13. Li Z, Wang H, Zhang P, Hui P, Huang J, Liao J, Zhang J, Bu J (2021) Live-streaming fraud detection: a heterogeneous graph neural network approach. In: *Proceedings of the ACM SIGKDD conference on knowledge discovery and data mining*, pp 3670–3678
 14. Liu C, Sun L, Ao X, Feng J, He Q, Yang H (2021) Intention-aware heterogeneous graph attention networks for fraud transactions detection. In: *Proceedings of the ACM SIGKDD conference on knowledge discovery and data mining*, pp 3280–3288
 15. Tian Y, Liu G (2023) Transaction fraud detection via spatial-temporal-aware graph transformer. arXiv preprint [arXiv:2307.05121](https://arxiv.org/abs/2307.05121)
 16. Yin H, Zhang Z, Wang Z, Özyurt Y, Liang W, Dong W, Zhao Y, Shan Y (2022) Behavioral graph fraud detection in e-commerce. In: *IEEE international conference on data mining workshops*, pp 1–8
 17. Khan S, Devlen C, Manno M, Hou D (2024) Mouse dynamics behavioral biometrics: a survey. *ACM Comput Surv* 56(6):1–33
 18. Feher C, Elovici Y, Moskovitch R, Rokach L, Schlar A (2012) User identity verification via mouse dynamics. *Inf Sci* 201:19–36
 19. Hu T, Niu W, Zhang X, Liu X, Lu J, Liu Y (2019) An insider threat detection approach based on mouse dynamics and deep learning. *Secur Commun Netw* 2019(1):3898951
 20. Awad A (2017) Collective framework for fraud detection using behavioral biometrics. *Inf Secur Pract Emerg Threats Perspect* 29–37
 21. Almalki S, Chatterjee P, Roy K (2019) Continuous authentication using mouse clickstream data analysis. In: *Security, privacy, and anonymity in computation, communication, and storage*, pp 76–85
 22. Maes S, Tuyls K, Vanschoenwinkel B, Manderick B (2002) Credit card fraud detection using bayesian and neural networks. In: *Proceedings of the international Naiso congress on neuro fuzzy technologies*, vol 261, p 270
 23. Ngai EW, Hu Y, Wong YH, Chen Y, Sun X (2011) The application of data mining techniques in financial fraud detection: a classification framework and an academic review of literature. *Decis Support Syst* 50(3):559–569
 24. Dhanalakshmi S, Subramanian C (2014) An analysis of data mining applications for fraud detection in securities market. *Int J Data Min Tech Appl* 3(1):1–9
 25. Ravisankar P, Ravi V, Rao GR, Bose I (2011) Detection of financial statement fraud and feature selection using data mining techniques. *Decis Support Syst* 50(2):491–500
 26. Glancy FH, Yadav SB (2011) A computational model for financial reporting fraud detection. *Decis Support Syst* 50(3):595–601
 27. Sithic HL, Balasubramanian T (2013) Survey of insurance fraud detection using data mining techniques. arXiv preprint [arXiv:1309.0806](https://arxiv.org/abs/1309.0806)
 28. Schiller J (2006) The impact of insurance fraud detection systems. *J Risk Insur* 73(3):421–438
 29. Zhang G, Li Z, Huang J, Wu J, Zhou C, Yang J, Gao J (2022) efraudcom: an e-commerce fraud detection system via competitive graph neural networks. *ACM Trans Inf Syst* 40(3):1–29
 30. Wang S, Liu C, Gao X, Qu H, Xu W (2017) Session-based fraud detection in online e-commerce transactions using recurrent neural networks. In: *Machine learning and knowledge discovery in databases: european conference*, pp 241–252. Springer
 31. Li L, Liu Z, Chen C, Zhang Y-L, Zhou J, Li X (2019) A time attention based fraud transaction detection framework. arXiv preprint [arXiv:1912.11760](https://arxiv.org/abs/1912.11760)
 32. Rao SX, Han Z, Yin H, Jiang J, Zhang Z, Zhao Y, Shan Y (2024) Fraud detection at ebay. *Emerg Markets Rev* 1–33
 33. Su X, Xue S, Liu F, Wu J, Yang J, Zhou C, Hu W, Paris C, Nepal S, Jin D et al (2022) A comprehensive survey on community detection with deep learning. *IEEE Trans Neural Netw Learn Syst* 35(4):4682–4702
 34. Weng H, Li Z, Ji S, Chu C, Lu H, Du T, He Q (2018) Online e-commerce fraud: a large-scale detection and analysis. In: *IEEE International conference on data engineering*, pp 1435–1440. IEEE
 35. Min W, Liang W, Yin H, Wang Z, Li M, Lal A (2021) Explainable deep behavioral sequence clustering for transaction fraud detection. arXiv preprint [arXiv:2101.04285](https://arxiv.org/abs/2101.04285)
 36. Ma J, Zhang D, Wang Y, Zhang Y, Pozdnoukhov A (2018) Graphrad: a graph-based risky account detection system. In: *Proceedings of the ACM SIGKDD conference on knowledge discovery and data mining*, vol 9, pp 1–9
 37. Aras MT, Guvensan MA (2023) A multi-modal profiling fraud-detection system for capturing suspicious airline ticket activities. *Appl Sci* 13(24):13121
 38. Park J, Kim M-H, Choi D-G (2021) Correspondence learning for deep multi-modal recognition and fraud detection. *Electronics* 10(7):800
 39. Park J, Kim M-H, Choi S, Kweon IS, Choi D-G (2019) Fraud detection with multi-modal attention and correspondence learning. In: *International conference on electronics, information, and communication*, pp 1–7. IEEE
 40. Wang G, Ma J, Chen G (2023) Attentive statement fraud detection: distinguishing multimodal financial data with fine-grained attention. *Decis Support Syst* 167:113913
 41. Rodrigues VF, Policarpo LM, Silveira DE, Rosa Righi R, Costa CA, Barbosa JLV, Antunes RS, Scorsatto R, Arcot T (2022) Fraud detection and prevention in e-commerce: a systematic literature review. *Electron Commer Res Appl* 56:101207
 42. Zheng N, Paloski A, Wang H (2011) An efficient user verification system via mouse movements. In: *Proceedings of the ACM conference on computer and communications security*, pp 139–150
 43. Mondal S, Bours P (2013) Continuous authentication using mouse dynamics. In: *International conference of the BIOSIG special interest group*, pp 1–12. IEEE

44. Shen C, Chen Y, Guan X, Maxion RA (2017) Pattern-growth based mining mouse-interaction behavior for an active user authentication system. *IEEE Trans Dependable Secure Comput* 17(2):335–349
45. Tan YXM, Binder A, Roy A (2017) Insights from curve fitting models in mouse dynamics authentication systems. In: *IEEE conference on application, information and network security*, pp 42–47
46. Gao L, Lian Y, Yang H, Xin R, Yu Z, Chen W, Liu W, Zhang Y, Zhu Y, Xu S, et al (2020) Continuous authentication of mouse dynamics based on decision level fusion. In: *International wireless communications and mobile computing*, pp 210–214 . IEEE
47. Zheng N, Paloski A, Wang H (2016) An efficient user verification system using angle-based mouse movement biometrics. *ACM Trans Inf Syst Secur* 18(3):1–27
48. Chong P, Tan YXM, Guarnizo J, Elovici Y, Binder A (2018) Mouse authentication without the temporal aspect—what does a 2d-CNN learn? In: *IEEE security and privacy workshops*, pp 15–21 . IEEE
49. Chong P, Elovici Y, Binder A (2019) User authentication based on mouse dynamics using deep neural networks: a comprehensive study. *IEEE Trans Inf Forensics Secur* 15:1086–1101
50. Antal M, Fejér N (2020) Mouse dynamics based user recognition using deep learning. *Acta Universitatis Sapientiae, Informatica* 12(1):39–50
51. Lin W, Sun L, Zhong Q, Liu C, Feng J, Ao X, Yang H (2021) Online credit payment fraud detection via structure-aware hierarchical recurrent neural network. In: *Proceedings of the thirtieth international joint conference on artificial intelligence*, pp 3670–3676
52. Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser Ł, Polosukhin I (2017) Attention is all you need. *Adv Neural Inf Process Syst* 30
53. Stragapede G, Delgado-Santos P, Tolosana R, Vera-Rodriguez R, Guest R, Morales A (2023) Mobile keystroke biometrics using transformers. In: *IEEE international conference on automatic face and gesture recognition*, pp 1–6 . IEEE
54. Havrylovych M, Danylov V (2023) Research on hybrid transformer-based autoencoders for user biometric verification. *Syst Res Inf Technol* 3:42–53
55. Yoon J, Park J, Wagata K, Park H, Teoh ABJ (2023) Pretrained implicit-ensemble transformer for open-set authentication on multimodal mobile biometrics. In: *Proceedings of the ACM international conference on multimedia*, pp 5909–5922
56. Alasadi SA, Bhaya WS (2017) Review of data preprocessing techniques in data mining. *J Eng Appl Sci* 12(16):4102–4107
57. Szegedy C, Liu W, Jia Y, Sermanet P, Reed S, Anguelov D, Erhan D, Vanhoucke V, Rabinovich A (2015) Going deeper with convolutions. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*
58. Fawcett T (2006) An introduction to roc analysis. *Pattern Recogn Lett* 27(8):861–874
59. Kingma DP, Ba J (2014) Adam: a method for stochastic optimization. arXiv preprint [arXiv:1412.6980](https://arxiv.org/abs/1412.6980)